

Top 50 Most-Asked

Java Interview Prep — 2026 Edition · Learn & Excel

If you have 5 days before your interview, this is the file. Read it cover-to-cover in 2 hours. Print it. Walk in. These 50 questions account for roughly 70-80% of what gets asked across Indian tech interviews — Service MNCs, product companies, GCCs, and startups alike.

Every question follows the same shape:

- **The question** — phrased the way an interviewer actually says it
 - **Asked at** — companies known to use this question pattern in recent rounds
 - **30-second answer** — what you'd say first if you only had one breath
 - **Step-by-step thought process** — what to say *out loud* while you think
 - **Edge cases to mention** — the depth signals that separate strong candidates
 - **What NOT to say** — the lines that downgrade you to "junior"
 - **Common follow-up** — what they'll likely ask next
-

Section A — The Hot 10 (asked in 80%+ of Java interviews)

These come up so often that walking in without solid answers is the single biggest reason candidates get rejected. Master these before anything else in the pack.

Q1 · HashMap internals — what happens when you call put()?

Asked at: TCS · Infosys · Razorpay · Swiggy · almost every Java interview

Difficulty: Medium · **Topic:** Collections

The question: "Walk me through what happens internally when you call `put("key", "value")` on a `HashMap`."

The 30-second answer: Hash the key → bitwise-AND with `(capacity - 1)` to find a bucket → if empty, place entry; if not, walk the bucket's chain (linked list, or red-black tree if bucket has ≥ 8 nodes AND map size ≥ 64) → resize if `size > capacity × 0.75`.

Step-by-step thought process: 1. "First, `hashCode()` is called on the key, then a supplemental hash mixes the high bits down — this reduces collisions when capacity is small." 2. "Index is computed via

`(capacity - 1) & hash` — works because capacity is always a power of 2." 3. "If the bucket is empty, entry goes in. Otherwise we walk the existing chain — comparing each entry's key via `equals()` ." 4. "In Java 8+, when a single bucket has 8 or more nodes AND the table's capacity is ≥ 64 , that bucket is converted to a red-black tree — $O(\log n)$ lookups instead of $O(n)$." 5. "After insert, if `size > capacity × loadFactor` (default 0.75), the map resizes — capacity doubles, every entry rehashed."

Edge cases to mention: - Null keys are allowed and stored in bucket 0 - Two keys with the same hashCode end up in the same bucket but are distinguished via `equals()` — so a broken `equals()` / `hashCode()` contract causes silent overwrites or duplicates - Treeification requires BOTH conditions (bucket ≥ 8 AND table capacity ≥ 64) — when capacity is below 64, the map resizes instead of treeifying

What NOT to say: - "It's just a hashtable" — interviewer wants depth, not category labels - "It uses chaining" — true but stale; Java 8 added treeification, mention both - "Treeification happens at 8 entries" — wrong, it's 8 entries *in one bucket* + table capacity ≥ 64

Common follow-up: "What's different about `ConcurrentHashMap`'s `put()`?"

Q2 · String immutability — why is String immutable in Java?

Asked at: TCS · Infosys · Accenture · most fresher rounds

Difficulty: Easy · **Topic:** Strings

The question: "Why is String immutable in Java? Walk me through the design reasoning."

The 30-second answer: Five reasons — security (paths, URLs, class names can't be mutated after validation), thread-safety (no synchronization needed), hashCode caching (computed once, reused forever — critical for HashMap keys), String Pool (sharing requires immutability), and class loader integrity (class names passed to loaders can't change mid-load).

Step-by-step thought process: 1. "Security — String is used for filesystem paths, network URLs, classloader inputs. If mutable, validated-then-mutated attacks become trivial." 2. "Thread safety — multiple threads can share a String reference with zero synchronization, because the value can't change." 3. "HashMap performance — String's hashCode is cached on first call. If String were mutable, the cached hash would become stale, breaking every hash-based collection." 4. "String Pool — JVM interns string literals so identical strings share memory. Sharing only works if values can't be modified through one reference." 5. "Class loading — when `Class.forName("com.foo.Bar")` runs, the class name string flows through the loader. Mutability would let other code race to change it mid-load."

Edge cases to mention: - `String` is `final` so even subclassing can't break the contract - The `value` array is `private final` (Java 9+: `byte[]` instead of `char[]` for compact strings) - `StringBuilder/StringBuffer` give you mutability when you actually need it — use them in loops to avoid creating N intermediate strings

What NOT to say: - "Because Java just made it that way" — non-answer, downgrades you - "It's immutable for performance" — partial; performance is one of five reasons, not the main one

Common follow-up: "What's the difference between `==` and `.equals()` for Strings? When does the String Pool help vs hurt?"

Q3 · ArrayList vs LinkedList — which one when?

Asked at: Almost every Java interview, fresher to senior

Difficulty: Easy-Medium · **Topic:** Collections

The question: "When would you choose ArrayList over LinkedList, and when the reverse?"

The 30-second answer: ArrayList = backed by an array, $O(1)$ random access, $O(n)$ insert/delete in the middle. LinkedList = doubly-linked nodes, $O(n)$ random access, $O(1)$ insert/delete if you already have the node reference. Default to ArrayList unless you're doing heavy middle-of-list mutations with iterators.

Step-by-step thought process: 1. "ArrayList wraps an Object[]. Random access via index is one memory lookup — $O(1)$. Adding to the end is amortized $O(1)$; when capacity fills, the array is doubled." 2. "Insert or delete in the middle of an ArrayList requires shifting every subsequent element — $O(n)$." 3. "LinkedList is a doubly-linked list. Random access requires walking from head or tail to the index — $O(n)$." 4. "Insert/delete in LinkedList is $O(1)$ only if you have the node reference (e.g. via an iterator). If you have just an index, you still walk $O(n)$ to find the node." 5. "In practice, ArrayList wins almost always. CPU cache locality favors arrays. LinkedList's per-node memory overhead (prev + next pointers) hurts on large datasets."

Edge cases to mention: - LinkedList is also a Deque — useful if you need queue + stack semantics from one collection - ArrayList's `remove(int)` is $O(n)$; `remove(Object)` is also $O(n)$ because it does a linear scan first - For large insert-heavy workloads, `ArrayDeque` often outperforms both

What NOT to say: - "LinkedList is faster for inserts" — true only with cached node references; misleading as a blanket claim - "Just use Vector for thread safety" — Vector is legacy; use `Collections.synchronizedList()` or `CopyOnWriteArrayList` instead

Common follow-up: "What about CopyOnWriteArrayList? When does that make sense?"

Q4 · == vs equals() vs the hashCode contract

Asked at: Razorpay · Swiggy · most product cos · senior-to-fresher

Difficulty: Medium · **Topic:** OOP fundamentals

The question: "Explain the difference between `==` and `.equals()`. Then explain the hashCode contract."

The 30-second answer: `==` compares references (same object in memory?). `.equals()` compares logical equality (same content). The contract: if `a.equals(b)`, then `a.hashCode() == b.hashCode()` — always. The reverse is NOT required: two objects with the same hashCode can have `equals()` return false. Breaking this contract corrupts HashMap, HashSet, and every hash-based collection.

Step-by-step thought process: 1. "`==` on objects is identity equality — are these two references pointing to the exact same memory location?" 2. "`.equals()` defaults to `==` in Object, but most classes override it to compare logical content (Strings compare characters, Integers compare values)." 3. "The hashCode contract: equal objects (per equals) MUST return the same hashCode. The reverse is not required — collisions are allowed." 4. "If you override `equals()` without overriding `hashCode()`, your objects break HashMap. You'd add a key, then `get()` would return null because the lookup hash points to a different bucket than the stored hash." 5. "The standard implementation: derive hashCode from the same fields used in equals — usually `Objects.hash(field1, field2, ...)`."

Edge cases to mention: - `Objects.equals(a, b)` is null-safe — use it instead of `a.equals(b)` when either side might be null - Records (standardized Java 16, preview from Java 14) auto-generate equals/hashCode based on all components — one big reason to use records for data classes - For mutable fields used in equals/hashCode: never mutate after adding to a hash collection; it corrupts the collection

What NOT to say: - "`equals()` checks if two objects are the same" — vague, says nothing - "Override `equals()` but not `hashCode()` — Java will figure it out" — actively wrong, will be silently broken

Common follow-up: "Why must you use `Objects.hash()` instead of just adding hashCodes?"

Q5 · volatile vs synchronized vs atomic — when each?

Asked at: Razorpay · Cred · senior backend rounds

Difficulty: Medium-Hard · **Topic:** Concurrency

The question: "What's the difference between `volatile`, `synchronized`, and `AtomicInteger`? Give me a use case where each is the right pick."

The 30-second answer: `volatile` = visibility only (writes flush to main memory, reads from main memory), no atomicity. `synchronized` = mutual exclusion AND visibility, allows compound actions. `Atomic*` = lock-free atomic operations (CAS) for single variables — faster than synchronized for simple counters and flags.

Step-by-step thought process: 1. "`volatile` guarantees that a write to the variable is visible to all threads immediately — no thread-local caching. But it does NOT make compound operations atomic. `volatile int counter; counter++;` is still a race condition." 2. "`synchronized` blocks (or methods) acquire a monitor on an object. Only one thread executes inside the block at a time. Also has visibility guarantees — exiting a synchronized block flushes writes, entering one reads fresh values." 3.

" `AtomicInteger` , `AtomicLong` , etc. use Compare-And-Swap (CAS) hardware instructions. Lock-free, much faster than synchronized for simple operations like `incrementAndGet()` ." 4. "Use volatile for: status flags read by many threads, written rarely (like `shutdownRequested`)." 5. "Use synchronized for: compound operations on shared state (check-then-act, multi-field updates)." 6. "Use Atomic for: counters, flags with read-modify-write semantics, where contention is moderate."

Edge cases to mention: - `volatile` ensures happens-before relationship — useful for double-checked locking on singletons (`volatile` on the instance field) - `synchronized` on `this` and on a class's `class` object are different locks — easy to confuse - For very-high-contention counters, `LongAdder` outperforms `AtomicLong` (striped internal state)

What NOT to say: - " `volatile` makes operations atomic" — common mistake, wrong - "Just use synchronized everywhere, it's safer" — performance-killing advice, gets you marked as junior

Common follow-up: "What's the double-checked locking pattern, and why does it need volatile?"

Q6 · Checked vs unchecked exceptions — when to use which?

Asked at: Most Java interviews, fresher onwards

Difficulty: Easy-Medium · **Topic:** Exception handling

The question: "Difference between checked and unchecked exceptions? When would you create a checked exception vs an unchecked one in your own code?"

The 30-second answer: Checked extends `Exception` (not `RuntimeException`), must be declared or caught — compiler enforces. Unchecked extends `RuntimeException` , no enforcement. Use checked for recoverable conditions the caller MUST handle (e.g. file not found, network unavailable). Use unchecked for programming errors and conditions the caller can't reasonably recover from (e.g. null argument, illegal state).

Step-by-step thought process: 1. "Java's exception hierarchy: `Throwable` → `Error` or `Exception`. `Exception` → `RuntimeException` (unchecked) or all others (checked)." 2. "Checked exceptions force the caller to handle or propagate at compile time — `throws` declaration required." 3. "Unchecked exceptions can be thrown anywhere with no declaration. `RuntimeException`, `NullPointerException`, `IllegalArgumentException` are all unchecked." 4. "When designing my own API: if the caller can REASONABLY recover (retry the network call, ask user for a different file path), make it checked. If it's a bug in the caller (passed null, called methods out of order), make it unchecked." 5. "In practice, modern Java (Spring, etc.) leans heavily toward unchecked — checked exceptions force boilerplate that often gets caught-and-ignored or wrapped meaninglessly. Spring even wraps JDBC's checked `SQLExceptions` into unchecked `DataAccessException`."

Edge cases to mention: - Errors (`OutOfMemoryError`, `StackOverflowError`) are not exceptions you should catch — they signal JVM-level problems - Lambda expressions can't throw checked exceptions unless the functional interface declares them — a common pain point with `Stream.map()` - Try-with-resources auto-closes `AutoCloseable` resources and propagates suppressed exceptions correctly

What NOT to say: - "Checked exceptions are bad, never use them" — opinionated and absolute; nuanced answer is better - "Just catch Exception everywhere" — anti-pattern, hides real bugs

Common follow-up: "Why doesn't `Stream.map()` let you throw checked exceptions easily?"

Q7 · JDK vs JRE vs JVM — what's the difference?

Asked at: TCS · Infosys · most fresher rounds

Difficulty: Easy · **Topic:** Java platform

The question: "Explain the difference between JDK, JRE, and JVM."

The 30-second answer: JVM = runtime engine that executes bytecode. JRE = JVM + standard libraries (everything needed to RUN Java apps). JDK = JRE + development tools like `javac`, `jdb`, `jar` (everything needed to BUILD Java apps).

Step-by-step thought process: 1. "JVM (Java Virtual Machine) is the abstract specification of a stack-based machine that executes Java bytecode. HotSpot, OpenJ9, GraalVM are concrete implementations." 2. "JRE (Java Runtime Environment) packages a JVM together with the standard Java class library — `java.lang`, `java.util`, `java.io`, etc." 3. "JDK (Java Development Kit) packages a JRE plus development tools: `javac` (compiler), `jar` (packager), `jdb` (debugger), `javadoc`, plus internal APIs for tools." 4. "Oracle stopped shipping a standalone JRE distribution starting with Java 11. You typically install a JDK and use `jlink` to build a slim runtime image if you need a JRE-equivalent." 5. "For a developer's mental model: write `.java` → compile with `javac` (JDK) → produces `.class` bytecode → executed by JVM (JRE)."

Edge cases to mention: - The JVM doesn't only run Java — Kotlin, Scala, Clojure, Groovy all compile to JVM bytecode - Multiple JVM implementations exist: HotSpot (Oracle), OpenJ9 (IBM/Eclipse), GraalVM (polyglot, ahead-of-time compilation) - Since Java 9, modules let you build a custom JRE containing only the modules your app uses — smaller runtime footprint

What NOT to say: - "JDK runs Java programs" — wrong; JVM runs them. JDK builds them. - "JRE is needed at runtime" — true historically, but Java 9+ blurs this with `jlink`

Common follow-up: "What does `javac` actually produce? What's bytecode?"

Q8 · final vs finally vs finalize — three different things

Asked at: TCS · Infosys · Cognizant · classic fresher question

Difficulty: Easy · **Topic:** Java language

The question: "What's the difference between `final`, `finally`, and `finalize`?"

The 30-second answer: `final` = keyword for immutability (final variable, final method, final class). `finally` = block that always runs after try/catch (for cleanup). `finalize()` = legacy Object method called by GC before reclaiming an object — deprecated since Java 9, deprecated-for-removal since Java 18 (still present in Java 21 but treat it as gone).

Step-by-step thought process: 1. "`final` has three uses: final variable = can't reassign after init, final method = can't override in subclass, final class = can't extend." 2. "`finally` is the try/catch/finally block — code inside finally runs whether the try block completes normally, throws, or returns early. Used for cleanup: closing resources, releasing locks." 3. "`finalize()` was a method on Object — the GC would call it before reclaiming an object, giving the object a chance to clean up. It was deprecated in Java 9 and marked deprecated-for-removal in Java 18 (JEP 421), and is still present in Java 21 — but treat it as gone." 4. "Modern equivalent of finalize: implement `AutoCloseable` and use try-with-resources, OR use `Cleaner` (Java 9+) which decouples cleanup logic from the object lifecycle."

Edge cases to mention: - `final` parameters can't be reassigned inside the method — useful for closures, prevents accidental mutation - `try-with-resources` is the preferred pattern over manual `finally { close(); }` — handles suppressed exceptions correctly - Even before Java 18, `finalize()` was unreliable — the GC made no guarantees about WHEN or IF it would be called

What NOT to say: - "Finalize is called when an object is destroyed" — vague and outdated; GC reclaims, doesn't "destroy" - Confusing them with one another — interviewer will downgrade you to fresher-tier on this question

Common follow-up: "How would you replace `finalize()` in modern Java?"

Q9 · Singleton pattern — 4 ways to implement it, which is thread-safe?

Asked at: Common at all levels

Difficulty: Medium · **Topic:** Design patterns + concurrency

The question: "Show me how to implement a Singleton in Java. Walk me through the trade-offs."

The 30-second answer: Four common ways — eager init (simple, thread-safe via class loading), lazy init with synchronized method (thread-safe but slow), double-checked locking with volatile (thread-safe and fast), enum (Bloch's preferred — handles serialization and reflection attacks). Enum is the cleanest for most cases.

Step-by-step thought process: 1. "Eager initialization: `private static final Singleton INSTANCE = new Singleton();` — thread-safe because class loading is thread-safe per the JLS. Downside: instantiated even if never used." 2. "Lazy with synchronized: method returns instance, creating it on first call. Thread-safe but every call pays synchronization cost." 3. "Double-checked locking: check instance == null without lock, lock only if null, check again inside lock. Faster than synchronized method but requires `volatile` on the instance field — otherwise the partially-constructed object can be visible to other threads." 4. "Enum singleton: `public enum Singleton { INSTANCE; }` — handles serialization automatically, prevents reflection attacks, lazy by default. Joshua Bloch's recommended

approach in *Effective Java*." 5. "For most code, enum is the right pick. Spring's `@Service` / `@Component` (default scope=singleton) handles this for managed beans."

Edge cases to mention: - Without `volatile` in DCL, the JIT can reorder writes — another thread might see a non-null reference to a not-yet-initialized object - Reflection can bypass private constructors — only the enum approach is truly bulletproof - Serialization can create a new instance via deserialization — enum handles this; other approaches need `readResolve()`

What NOT to say: - "Just use synchronized on getInstance" — works but slow, and shows you don't know DCL - "Singleton is an anti-pattern, never use it" — opinionated; the interviewer wants to see you can implement it correctly

Common follow-up: "Why does double-checked locking specifically require volatile?"

Q10 · Garbage collection — what are the major collectors and when each?

Asked at: Mid-to-senior backend rounds

Difficulty: Medium-Hard · **Topic:** JVM internals

The question: "Walk me through the major garbage collectors in modern Java. When would you choose each?"

The 30-second answer: G1 (default since Java 9) — balanced throughput and pause times, good for most server apps. ZGC and Shenandoah — ultra-low pause (sub-millisecond), for latency-sensitive systems. Parallel GC — throughput-optimized, batch jobs. Serial GC — single-threaded, small heaps only.

Step-by-step thought process: 1. "All major GCs are generational — heap split into young generation (Eden + 2 survivor spaces) and old generation. Most allocations happen in Eden; most objects die young." 2. "G1 (Garbage First) — default since Java 9. Divides heap into regions, prioritizes collecting regions with the most garbage. Targets predictable pause times (`MaxGCPauseMillis`). Good default for most apps." 3. "ZGC — concurrent, low-latency. Pause times under 1ms even on huge heaps (TB-scale). Trade-off: slightly lower throughput. Production-ready since Java 15." 4. "Shenandoah — similar low-pause design, developed at Red Hat. Comparable to ZGC for most workloads." 5. "Parallel GC — older, throughput-optimized, stop-the-world. Good for batch processing where pauses don't matter but total time does." 6. "Serial GC — single-threaded. Only for small heaps and embedded systems."

Edge cases to mention: - G1 has a humongous object threshold — objects $\geq \frac{1}{2}$ region size allocate directly in old gen - ZGC supports colored pointers (uses 64-bit pointer metadata) — required for its concurrent design - Java 21 introduced generational ZGC as opt-in (non-generational remained default in 21); generational became default in Java 24 — improved throughput either way

What NOT to say: - "Java's GC handles everything automatically, don't worry about it" — true for hello-world, wrong for production - "CMS is the best low-latency option" — CMS was REMOVED in Java 14, this dates you badly

Common follow-up: "How do you tune G1 for a Spring Boot app with bursty traffic?"

Section B — Collections Deep Cuts (asked when interviewer wants to go beyond basics)

These come up after you've nailed the Hot 10. Strong on these = "candidate knows their stuff."

Q11 · ConcurrentHashMap — how is it different from HashMap?

Asked at: Razorpay · Cred · senior backend rounds

Difficulty: Medium-Hard · **Topic:** Concurrency + Collections

The question: "How does ConcurrentHashMap differ from HashMap, and what changed between Java 7 and Java 8?"

The 30-second answer: HashMap is not thread-safe. ConcurrentHashMap is. Java 7 used segment-based locking (16 segments by default, each locked independently). Java 8 ditched segments — uses CAS for inserts on empty buckets, synchronizes on the first node of a bucket for collisions. Much higher concurrency in Java 8+.

Step-by-step thought process: 1. "HashMap is fast but unsafe for concurrent writes — you can get corrupted state, infinite loops on resize (classic Java 7 bug), or lost updates." 2. "ConcurrentHashMap (Java 7) divided the table into 16 segments, each with its own lock. Writers held only one segment's lock at a time, so up to 16 writers could proceed in parallel." 3. "Java 8 rewrote it: lock-free on empty buckets using `compareAndSwap`, fine-grained synchronization on the first node of a bucket when there's a collision. No more segments." 4. "Reads are completely lock-free in both versions — you can read while others write." 5. "`computeIfAbsent`, `merge`, `compute` are the atomic compound operations — use these instead of check-then-put."

Edge cases to mention: - ConcurrentHashMap forbids null keys and null values (HashMap allows both) — null would create ambiguity in get/contains return values - Iterators are weakly consistent — may or may not reflect modifications made after iteration started, but won't throw ConcurrentModificationException - For high-concurrency counts, prefer `mappingCount()` (returns `long`) over `size()` (returns `int`). Javadoc explicitly says the value is an estimate — exact-at-read is essentially impossible without global locking

What NOT to say: - "Just use Collections.synchronizedMap(new HashMap())" — that wraps with one giant lock, much slower than ConcurrentHashMap - "ConcurrentHashMap is always faster than HashMap" — false; HashMap is faster single-threaded

Common follow-up: "When would you use `computeIfAbsent` vs `putIfAbsent`?"

Q12 · Fail-fast vs fail-safe iterators

Asked at: Common in Indian product company senior rounds

Difficulty: Medium · **Topic:** Collections + iterators

The question: "What's the difference between fail-fast and fail-safe iterators? Give an example of each."

The 30-second answer: Fail-fast iterators throw `ConcurrentModificationException` if the underlying collection is modified during iteration (`ArrayList`, `HashMap` iterators). Fail-safe iterators work on a snapshot or use copy-on-write semantics, so concurrent modification is allowed (`ConcurrentHashMap`, `CopyOnWriteArrayList` iterators).

Step-by-step thought process: 1. "Most JCF collections (`ArrayList`, `HashMap`, `HashSet`) keep an internal `modCount` counter. The iterator captures `modCount` at creation; each `next()` checks it hasn't changed." 2. "If something modifies the collection (even via another iterator), `modCount` increments. The next `next()` call detects the mismatch and throws `ConcurrentModificationException`." 3. "This is fail-FAST — fails immediately to prevent silently inconsistent iteration." 4. "Fail-SAFE iterators (`ConcurrentHashMap`, `CopyOnWriteArrayList`, `ConcurrentLinkedQueue`) don't throw on concurrent modification. They either iterate over a snapshot (`CopyOnWriteArrayList`) or are weakly consistent (`ConcurrentHashMap`)." 5. "Important caveat: 'fail-safe' doesn't mean 'sees all updates' — you may miss modifications made during iteration."

Edge cases to mention: - `Iterator.remove()` is the only safe way to modify a fail-fast collection during iteration - Single-threaded code can hit `ConcurrentModificationException` too — if you call `list.remove(x)` directly inside an enhanced for-loop - `Collections.unmodifiableList` returns a wrapper — modification attempts throw immediately, not on next iteration

What NOT to say: - "Fail-safe is always better" — copy-on-write has heavy write cost; fail-fast is correct for single-threaded code - "ConcurrentHashMap iterators throw on modification" — wrong, they don't

Common follow-up: "When would `CopyOnWriteArrayList` actually be the right choice?"

Q13 · CopyOnWriteArrayList — when is it the right pick?

Asked at: Product cos · senior rounds

Difficulty: Medium · **Topic:** Concurrent collections

The question: "When would you actually use `CopyOnWriteArrayList`? What are the trade-offs?"

The 30-second answer: Use when reads vastly outnumber writes — listener lists, configuration data, observer patterns. Every write copies the entire underlying array ($O(n)$), but reads need zero synchronization. Iterators are guaranteed snapshot-consistent.

Step-by-step thought process: 1. "On every mutation (add, remove, set), `CopyOnWriteArrayList` creates a brand new copy of the underlying array with the change applied, then replaces the reference." 2. "Reads — including iteration — happen against whichever array the reference pointed to at read-time."

Zero synchronization, no locks." 3. "Iterators capture the array reference at creation, so they iterate a consistent snapshot — modifications during iteration don't affect them and don't throw." 4. "Cost: $O(n)$ per write, plus garbage from the discarded array. Reads are $O(1)$ lock-free." 5. "Sweet spot: small to medium lists, many readers (e.g. event listeners, plugin registries), few writers."

Edge cases to mention: - `iterator().remove()` throws `UnsupportedOperationException` — iterators are snapshot, can't reach back to mutate - For maps, `CopyOnWriteArrayMap` doesn't exist; use `ConcurrentHashMap` instead - If your collection has more than a few thousand elements with even moderate writes, this is the wrong choice — the copy cost dominates

What NOT to say: - "It's just a thread-safe `ArrayList`" — true but hides the copy-on-write cost - "Use it for queues" — use `ConcurrentLinkedQueue` or `ArrayBlockingQueue` for queues

Common follow-up: "How does `CopyOnWriteArraySet` relate to this?"

Q14 · TreeMap and red-black trees

Asked at: Common in mid-level rounds

Difficulty: Medium · **Topic:** Sorted collections

The question: "What's the underlying data structure of `TreeMap`, and what guarantees does it give you over `HashMap`?"

The 30-second answer: `TreeMap` is a self-balancing red-black tree. Operations are $O(\log n)$ instead of `HashMap`'s $O(1)$ average. In exchange: keys are kept in sorted order, and you get navigation methods (`floorKey`, `ceilingKey`, `headMap`, `tailMap`, `subMap`).

Step-by-step thought process: 1. "Red-black tree is a self-balancing binary search tree with five invariants — most importantly, no red node has a red parent, and every path from root to null has the same number of black nodes." 2. "These invariants ensure the tree stays approximately balanced — height is $O(\log n)$ — guaranteeing $O(\log n)$ for get/put/remove." 3. "Keys must be `Comparable` (or you supply a `Comparator` at construction). The tree uses comparison, not hash." 4. "Navigation methods are `TreeMap`'s killer feature: `firstKey`, `lastKey`, `floorKey(k)` (largest key $\leq k$), `ceilingKey(k)` (smallest key $\geq k$), `headMap(k)` (entries $< k$), etc." 5. "Use `TreeMap` when you need ordered iteration, range queries, or 'find closest key' operations. Use `HashMap` when you only need fast lookup and don't care about order."

Edge cases to mention: - `TreeMap` doesn't allow null keys (uses comparison; null can't be compared) but allows null values - `firstEntry`, `lastEntry`, `pollFirstEntry`, `pollLastEntry` are useful for priority-queue-like patterns - `ConcurrentSkipListMap` is the concurrent equivalent — also sorted, also $O(\log n)$, thread-safe

What NOT to say: - "TreeMap is faster than `HashMap`" — wrong, `HashMap` is faster for plain lookups - "TreeMap is a B-tree" — wrong, it's a red-black tree

Common follow-up: "How would you find all entries with keys between X and Y?"

Q15 · LinkedHashMap and LRU cache

Asked at: System design adjacent · mid-level rounds

Difficulty: Medium · **Topic:** Specialized collections

The question: "How would you implement an LRU cache in Java in 10 lines?"

The 30-second answer: Extend `LinkedHashMap` with `accessOrder=true` and override `removeEldestEntry()` to return true when size exceeds capacity. LinkedHashMap maintains a doubly-linked list of entries in access order; the eldest (least recently accessed) is at the head, ready to be evicted.

Step-by-step thought process: 1. "LinkedHashMap is HashMap + a doubly-linked list threading through all entries. Iteration order can be insertion order (default) or access order (constructor flag)." 2. "With `accessOrder=true`, every `get()` and `put()` moves the touched entry to the tail of the linked list. The head is always the least-recently-used." 3. "`removeEldestEntry(Map.Entry)` is called after every put. Default returns false. Override to return true when `size() > maxCapacity` — entry gets removed automatically." 4. "Result: a self-maintaining LRU cache with O(1) get/put/evict." 5. "Code: `new LinkedHashMap<K, V>(capacity, 0.75f, true) { protected boolean removeEldestEntry(Map.Entry<K,V> eldest) { return size() > maxCapacity; } }`"

Edge cases to mention: - Not thread-safe — wrap with `Collections.synchronizedMap` if needed (or use Caffeine library for production) - For production use, Caffeine (W-TinyLFU eviction) is dramatically better than LRU for most access patterns - Guava's CacheBuilder is also an option but Caffeine has largely superseded it

What NOT to say: - "Use HashMap + a separate list" — works but you have to maintain both manually, error-prone - "Just use Redis" — out-of-scope, interviewer wants a Java answer

Common follow-up: "What if you needed time-based expiration in addition to LRU?"

Q16 · HashSet vs TreeSet vs LinkedHashMap

Asked at: Fresher to mid-level

Difficulty: Easy-Medium · **Topic:** Sets

The question: "Compare HashSet, TreeSet, and LinkedHashMap."

The 30-second answer: HashSet — hash-based, O(1) ops, no order. TreeSet — red-black tree, O(log n) ops, sorted order. LinkedHashMap — HashSet + linked list, O(1) ops, insertion order. Pick HashSet by default; TreeSet for sorted; LinkedHashMap when you want predictable iteration.

Step-by-step thought process: 1. "HashSet wraps a HashMap internally — your set elements are the keys, with a dummy constant value. O(1) average for add/remove/contains." 2. "TreeSet wraps a TreeMap — same red-black tree, sorted order, O(log n) ops. Navigation methods (`first`, `last`, `floor`,

`ceiling` , `headSet` , `tailSet`) available." 3. "LinkedHashSet wraps LinkedHashMap — O(1) ops like HashSet, but iteration is in insertion order (or access order with the constructor flag)." 4. "Memory: LinkedHashSet has overhead per element (linked list pointers); TreeSet has tree-node overhead; HashSet is lightest." 5. "Default pick: HashSet. When you need sorted output: TreeSet. When you need stable iteration order (e.g. for testing, deterministic output): LinkedHashSet."

Edge cases to mention: - HashSet allows one null; TreeSet doesn't (uses comparison) - All three require correctly-implemented `equals()/hashCode()` (HashSet/LinkedHashSet) or `Comparable` / `Comparator` (TreeSet) - `EnumSet` is a specialized, bitmap-based set for enums — dramatically faster than HashSet for enum values

What NOT to say: - "TreeSet is faster because it's sorted" — wrong, $\log n > 1$ - "Use LinkedHashSet for thread safety" — none of these are thread-safe; use `ConcurrentSkipListSet` or `CopyOnWriteArraySet`

Common follow-up: "How do you make any of these thread-safe?"

Q17 · Comparable vs Comparator

Asked at: Common at all levels

Difficulty: Easy-Medium · **Topic:** Sorting

The question: "What's the difference between Comparable and Comparator? When would you use each?"

The 30-second answer: Comparable is implemented BY the class itself — defines the class's natural ordering via `compareTo()` . Comparator is a separate object — defines an alternative ordering, useful when you don't own the class or need multiple orderings.

Step-by-step thought process: 1. " `Comparable<T>` has one method: `int compareTo(T other)` . Returns negative if this < other, zero if equal, positive if this > other." 2. "You implement Comparable on a class to give it a natural ordering — Integer compares numerically, String compares lexicographically, your `Employee` class might compare by ID." 3. "Comparator is external. Useful when: (a) you don't control the class, (b) you need multiple different orderings (e.g. employees by name vs by salary), (c) you want ad-hoc sorting without modifying the class." 4. "Java 8 made Comparators dramatically nicer: `Comparator.comparing(Employee::getName).thenComparing(Employee::getSalary)` chains naturally." 5. " `reversed()` , `nullsFirst()` , `nullsLast()` are built-in modifiers."

Edge cases to mention: - The contract: `compare(a, b)` and `compare(b, a)` must have opposite signs; transitive ordering required - TreeSet and TreeMap silently misbehave if your Comparable is inconsistent with equals - Modern Java preference: define Comparable for natural ordering, use Comparator factory methods for everything else

What NOT to say: - "Comparable is for sorting, Comparator is for searching" — wrong, both are for ordering - Returning subtractions in compareTo (`return this.value - other.value`) — integer overflow risk; use `Integer.compare()` instead

Common follow-up: "How would you sort a List of Employees by department, then by salary descending?"

Q18 · Queue vs Deque vs Stack

Asked at: Common in mid-level rounds

Difficulty: Medium · **Topic:** Collections

The question: "When would you use a Deque vs a Queue vs a Stack in Java? Are Stack and Vector still relevant?"

The 30-second answer: `Stack` and `Vector` are legacy — synchronized, slow, avoid. For LIFO (stack semantics), use `Deque` (`ArrayDeque` recommended). For FIFO (queue), use `Queue` interface with `LinkedList` or `ArrayDeque`. For concurrent queues, use `BlockingQueue` implementations like `LinkedBlockingQueue` or `ArrayBlockingQueue`.

Step-by-step thought process: 1. "`Stack` extends `Vector` — both are from Java 1.0, both fully synchronized (slow), both deprecated in spirit if not officially." 2. "Modern preferred Stack: `Deque<T> stack = new ArrayDeque<>();` then use `push`, `pop`, `peek`." 3. "Modern preferred Queue: `Queue<T> q = new ArrayDeque<>();` or `new LinkedList<>();`. Use `offer`, `poll`, `peek`." 4. "For producer-consumer: `BlockingQueue` implementations — `ArrayBlockingQueue` (bounded), `LinkedBlockingQueue` (optionally bounded), `PriorityBlockingQueue`, `SynchronousQueue`. `put` / `take` block as needed." 5. "For lock-free concurrent queues: `ConcurrentLinkedQueue` (unbounded), `ConcurrentLinkedDeque`."

Edge cases to mention: - `ArrayDeque` is generally faster than `LinkedList` for both queue and stack ops (CPU cache locality) - `PriorityQueue` is a heap-backed priority queue — not FIFO, ordered by Comparator/Comparable - Java 21 added `SequencedCollection` interface — adds `getFirst`, `getLast`, `addFirst`, `addLast`, `reversed()` to ordered collections

What NOT to say: - "Use Stack class for LIFO" — legacy, marks you as not following modern Java - "LinkedList is the best Deque" — `ArrayDeque` outperforms it for almost all access patterns

Common follow-up: "How does `BlockingQueue` help in a producer-consumer setup?"

Q19 · Iterator vs ListIterator vs Spliterator

Asked at: Mid-level rounds

Difficulty: Medium · **Topic:** Iterators

The question: "What's a *Spliterator*, and how is it different from a regular *Iterator*?"

The 30-second answer: *Iterator* = sequential, forward-only, optional remove. *ListIterator* = bidirectional, can add/set/remove. *Spliterator* (Java 8+) = supports parallel processing via `trySplit()`, used internally by streams. It can describe characteristics of the underlying source (ORDERED, SIZED, IMMUTABLE, etc.) so the stream pipeline can optimize.

Step-by-step thought process: 1. "*Iterator*: `hasNext`, `next`, optional `remove`. One direction. Used by enhanced for-loop." 2. "*ListIterator* (lists only): adds `hasPrevious`, `previous`, `add`, `set`, `nextIndex`, `previousIndex`. Bidirectional, mutating." 3. "*Spliterator*: designed for parallel traversal. `tryAdvance(action)` processes one element. `trySplit()` returns a NEW *Spliterator* covering a portion of the source — the original now covers the rest." 4. "`characteristics()` returns a bitmask: ORDERED, SIZED, SUBSIZED, IMMUTABLE, CONCURRENT, NONNULL, DISTINCT, SORTED. Stream operations check these to pick optimal execution strategies." 5. "You rarely use *Spliterator* directly — `Collection.stream()` builds one for you. But if you write a custom collection, you might implement one to get good stream performance."

Edge cases to mention: - `Spliterator.estimateSize()` returns the estimated element count if SIZED; otherwise an estimate - Custom *Spl iterators* are essential for IO-bound or generator-style streams (e.g. wrapping `BufferedReader.lines()`) - `Stream.iterate(seed, hasNext, next)` (Java 9+) is a friendlier way to build streams that previously needed custom *Spl iterators*

What NOT to say: - "*Spliterator* is just a parallel iterator" — incomplete; the characteristics flags are the real win - "Always use parallel streams for performance" — overhead often exceeds benefits for small or non-CPU-bound work

Common follow-up: "When does `parallelStream()` actually outperform `stream()`?"

Q20 · Map.Entry, computeIfAbsent, and the Java 8 Map enhancements

Asked at: Mid-level + senior rounds

Difficulty: Medium · **Topic:** Collections + idiomatic Java

The question: "What new methods were added to *Map* in Java 8, and when would you use `computeIfAbsent` over `putIfAbsent`?"

The 30-second answer: Java 8 added `getOrDefault`, `putIfAbsent`, `compute`, `computeIfAbsent`, `computeIfPresent`, `merge`, `replace`, `replaceAll`, `forEach`. Use `computeIfAbsent` when you need to LAZILY compute the value (expensive call). Use `putIfAbsent` when you already have the value ready.

Step-by-step thought process: 1. "`getOrDefault(key, default)` — replaces the `map.containsKey(k) ? map.get(k) : default` pattern." 2. "`putIfAbsent(k, v)` — puts only if key absent. Returns existing value if present, null otherwise." 3. "`computeIfAbsent(k, key -> expensiveCompute(key))` — same idea but the value is

computed by a function ONLY if the key is absent. Avoids wasting work." 4. " `compute(k, (k, oldV) -> newV)` — always invokes the function, even if oldV is null. Useful for counters: `map.compute(word, (k, v) -> v == null ? 1 : v + 1)` ." 5. " `merge(k, value, BiFunction)` — if absent, put value; if present, apply function to (oldValue, value). Cleanest counter increment: `map.merge(word, 1, Integer::sum)` ."

Edge cases to mention: - `computeIfAbsent` is atomic on ConcurrentHashMap — the function is invoked at most once per key - `compute` / `merge` returning null deletes the entry - `forEach((k, v) -> ...)` replaces the verbose `entrySet().iterator()` loop

What NOT to say: - "putIfAbsent and computeIfAbsent are the same" — wrong; eager vs lazy value construction is the key distinction - "Just use containsKey + get" — works but slow on ConcurrentHashMap (two lookups) and not atomic

Common follow-up: "How would you build a word-count Map with one line of Java?"

Section C — Multithreading Traps (where mid-level candidates fail)

These are the questions where "I know threads" candidates get separated from candidates who actually understand concurrency.

Q21 · ThreadLocal — what it is and the classic memory leak

Asked at: Razorpay · Cred · senior backend

Difficulty: Medium-Hard · **Topic:** Concurrency

The question: "What's ThreadLocal? Why does it cause memory leaks in thread pool environments?"

The 30-second answer: ThreadLocal stores a value per thread — each thread sees its own copy. Memory leak: in a thread pool, threads are reused. ThreadLocal values aren't cleared between tasks. Over time, ThreadLocal entries accumulate per thread, and the GC can't reclaim them because the thread still holds a reference.

Step-by-step thought process: 1. " `ThreadLocal<T>` is conceptually a map from `Thread` → `T`. Each thread has its own `ThreadLocalMap`, accessed via the `ThreadLocal` instance as the key." 2. "Use cases: per-request context (security principal, transaction ID) without passing it through every method signature; thread-confined caches; `SimpleDateFormat` instances (since `SimpleDateFormat` isn't thread-safe)." 3. "The leak: in a thread pool (Tomcat, `ExecutorService`), thread instances are reused for many tasks. If a task sets a `ThreadLocal` value and doesn't remove it, the value sticks to that thread forever." 4. "Even worse — `ThreadLocalMap`'s entries use weak references for the KEY (the `ThreadLocal` instance) but strong for the VALUE. If the `ThreadLocal` goes out of scope, the key becomes weakly reachable, but the value is still strongly held." 5. "Fix: always pair `threadLocal.set(value)` with

`threadLocal.remove()` in a finally block, OR use Java 21's `ScopedValue` for the same use case without the leak risk."

Edge cases to mention: - `InheritableThreadLocal` copies values to child threads at creation time — usually NOT what you want with thread pools - `ScopedValue` (Java 21) is the modern replacement — immutable, scoped to a `run` block, automatically cleared - Major frameworks (Spring, SLF4J MDC) have `ThreadLocal` cleanup as a common bug source

What NOT to say: - "ThreadLocal is just thread-safe storage" — true but misses the lifecycle/leak issue - "Just don't use ThreadLocal" — too absolute; it has valid uses if you're disciplined

Common follow-up: "How does Java 21's `ScopedValue` fix these issues?"

Q22 · Deadlock — how does it happen, how do you detect/avoid it?

Asked at: Common in mid-to-senior rounds

Difficulty: Medium-Hard · **Topic:** Concurrency

The question: "Walk me through a classic deadlock scenario and tell me how you'd prevent it."

The 30-second answer: Deadlock requires four conditions (Coffman's): mutual exclusion, hold-and-wait, no preemption, circular wait. Classic scenario: Thread A locks resource 1 then tries for 2; Thread B locks 2 then tries for 1. Avoidance: enforce a global lock ordering (always acquire resources in the same order), use `tryLock` with timeout, or use higher-level constructs (concurrent collections).

Step-by-step thought process: 1. "Coffman's four conditions — all four must hold for deadlock. Break any one and deadlock is impossible." 2. "Classic two-thread, two-lock deadlock: A holds L1 wants L2; B holds L2 wants L1. Both block forever." 3. "Detection: jstack on a running JVM shows BLOCKED threads with their wait targets. Java even logs 'Found one Java-level deadlock' when threads hold mutual locks." 4. "Prevention #1 — lock ordering: define a global order for all locks (e.g. by ID). Always acquire in ascending order. Breaks the circular-wait condition." 5. "Prevention #2 — `Lock.tryLock(timeout)` instead of unbounded acquire. If timeout fires, release everything you hold and retry." 6. "Prevention #3 — avoid holding locks across calls to unknown code (especially callbacks, lambdas). You don't know what locks the callee might try to acquire."

Edge cases to mention: - Livelock: threads aren't blocked but keep retrying and getting nowhere (e.g. two polite people stepping aside in a corridor) - Starvation: a thread never gets a chance to run because higher-priority threads dominate — not technically deadlock but adjacent problem - Synchronized blocks acquire a single monitor; you can't selectively unlock them — `ReentrantLock` gives more control

What NOT to say: - "Just don't use locks" — sometimes you must; advice is unhelpful - "Increase thread count to avoid deadlock" — actually makes it worse

Common follow-up: "How do you detect deadlocks in a production system without taking a thread dump?"

Q23 · CompletableFuture — chaining async work without callback hell

Asked at: Cred · Razorpay · Swiggy · backend rounds

Difficulty: Medium-Hard · **Topic:** Async + concurrency

The question: "Show me how you'd use `CompletableFuture` to call three services in parallel and combine the results."

The 30-second answer: `CompletableFuture.supplyAsync(call1)`, similar for `call2` and `call3`, then `CompletableFuture.allOf(f1, f2, f3).thenApply(v -> combineResults(f1.join(), f2.join(), f3.join()))`. Use `thenCompose` to chain dependent async calls (`flatMap`), `thenCombine` to combine two independent ones.

Step-by-step thought process: 1. "`CompletableFuture.supplyAsync(supplier)` runs the supplier on the `ForkJoinPool` (or specified `Executor`) and returns a future." 2. "For parallel-then-combine: kick off all three with `supplyAsync`, get three futures. `allOf` returns a `CompletableFuture<Void>` that completes when all complete. After that, `.join()` on each gives the result." 3. "For chaining (B depends on A's result): `futureA.thenCompose(a -> callB(a))`. `thenCompose` is `flatMap`; `thenApply` is `map`." 4. "For combining two independent ones: `futureA.thenCombine(futureB, (a, b) -> ...)`." 5. "For fallback on error: `.exceptionally(throwable -> fallbackValue)` or `.handle((value, throwable) -> ...)`."

Edge cases to mention: - Default executor is the common `ForkJoinPool` — if your async work is IO-bound and blocks, you can starve the pool. Pass a dedicated `Executor`. - `.join()` is unchecked; `.get()` throws checked `InterruptedException` + `ExecutionException` - Java 21 virtual threads + structured concurrency (preview) is the new shiny — often a cleaner alternative

What NOT to say: - "Use `Future.get()` in a loop" — defeats the purpose; you're blocking - "Use `RxJava`" — out of scope unless asked; `CompletableFuture` is the question

Common follow-up: "How does this change with Java 21 virtual threads?"

Q24 · ExecutorService and the dangers of newFixedThreadPool

Asked at: Senior backend rounds

Difficulty: Medium · **Topic:** Concurrency

The question: "What's wrong with `Executors.newFixedThreadPool(10)` in production?"

The 30-second answer: It uses an unbounded `LinkedBlockingQueue` for tasks. If submission rate > processing rate, the queue grows unboundedly — eventually OOM. Production code should construct `ThreadPoolExecutor` directly with a bounded queue and a sensible `RejectedExecutionHandler`.

Step-by-step thought process: 1. "`Executors.newFixedThreadPool(10)` is a convenience method. Under the hood: `new ThreadPoolExecutor(10, 10, 0, MILLIS, new LinkedBlockingQueue<Runnable>())`." 2. "The `LinkedBlockingQueue` with no capacity argument is unbounded. If tasks arrive faster than 10 threads can process them, the queue fills with millions of tasks before anything fails — `OutOfMemoryError`, with confused stack traces." 3. "`newCachedThreadPool()` is worse — it creates unlimited threads, can OOM faster." 4. "Production: construct `ThreadPoolExecutor` directly. Pick a bounded queue (e.g. `new ArrayBlockingQueue<>(1000)`) and a `RejectedExecutionHandler` — `CallerRunsPolicy` (pushes back on submitter), `AbortPolicy` (throws), or your own that drops to a dead-letter queue." 5. "Pre-compute pool size: CPU-bound ~ `Runtime.getRuntime().availableProcessors()`; IO-bound ~ much higher; mixed = profile."

Edge cases to mention: - Spring's `@Async` defaults to a `SimpleAsyncTaskExecutor` (new thread per task!) unless configured. Always configure a `TaskExecutor` bean. - Java 21 virtual threads change the calculus — for IO-bound work, `Executors.newVirtualThreadPerTaskExecutor()` is now sensible - `ThreadFactory` lets you set thread names — critical for debugging stack dumps

What NOT to say: - "newFixedThreadPool is the standard choice" — popular but production-dangerous - "Just use more threads" — backpressure, not threads, is the answer

Common follow-up: "How does this change with Java 21 virtual threads?"

Q25 · synchronized — the difference between method-level and block-level

Asked at: Common at all levels

Difficulty: Medium · **Topic:** Synchronization

The question: "What's the difference between `synchronized` on a method versus a `synchronized` block?"

The 30-second answer: `synchronized` on an instance method = locks on `this`. `synchronized` on a static method = locks on the Class object. `synchronized(obj) { ... }` block = locks on `obj`. Block-level is preferred because it lets you (a) lock on a private dedicated object (no external code can lock it), (b) shrink the critical section to just the code that needs it.

Step-by-step thought process: 1. "`public synchronized void m() { ... }` is equivalent to `public void m() { synchronized(this) { ... } }`." 2. "`public static synchronized void m() { ... }` locks on `MyClass.class` — the Class object." 3. "`synchronized(this)` is dangerous in libraries — external code can also acquire `this` as a lock, causing deadlocks you can't predict." 4. "Best practice: declare a private final Object: `private final Object lock = new Object();` and use `synchronized(lock) { ... }`. No one outside can lock it." 5. "Block scope lets you shrink the critical section to JUST the racy part. Method-level synchronization is often coarser than necessary, hurting throughput."

Edge cases to mention: - `synchronized` is reentrant — same thread can re-acquire the same lock without deadlocking itself - `ReentrantLock` offers `tryLock`, fairness, interruptible acquisition, and condition variables — capabilities `synchronized` doesn't have - Locking on a boxed Long/Integer/String is unsafe — the JVM may intern values, you might be locking on the same object as unrelated code

What NOT to say: - "synchronized is always slow, avoid it" — modern JVMs have biased locking and lightweight locking; uncontended synchronized is fast - "Lock on `Class.class` for static methods" — works but unusual; the implicit lock is the same Class object

Common follow-up: "When would you use `ReentrantLock` over `synchronized`?"

Q26 · Producer-consumer with `BlockingQueue`

Asked at: Common in design rounds

Difficulty: Medium · **Topic:** Concurrency

The question: "Design a producer-consumer system in Java. How would you handle backpressure?"

The 30-second answer: Use a bounded `BlockingQueue` between producers and consumers. Producers call `put()` (blocks if full); consumers call `take()` (blocks if empty). The bounded queue provides natural backpressure — when full, producers slow down.

Step-by-step thought process: 1. "`BlockingQueue<Task>` interface; common impl: `ArrayBlockingQueue<>(capacity)` for bounded, `LinkedBlockingQueue` for optionally-bounded." 2. "Producer threads: `queue.put(task)` — blocks if the queue is full. Naturally throttles producers when consumers can't keep up. This is backpressure." 3. "Consumer threads: `queue.take()` — blocks if empty. No busy-wait, no polling." 4. "Bounded is critical — unbounded queues just defer the problem until OOM." 5. "For shutdown: poison-pill pattern (insert a sentinel object consumers check for and exit on), or use `ExecutorService` + future for clean shutdown."

Edge cases to mention: - `offer(task, timeout)` and `poll(timeout)` are the non-blocking variants with timeout — useful when you need to log "queue full, dropping" instead of blocking - For very-high throughput: Disruptor library (LMAX) outperforms `BlockingQueue` by orders of magnitude - `SynchronousQueue` has zero capacity — every put must directly match a take, useful for handoff patterns

What NOT to say: - "Use `wait()` and `notifyAll()`" — outdated; `BlockingQueue` is the modern idiom - "Use a List and synchronize on it" — reinventing the wheel badly

Common follow-up: "What if you needed different priority levels for tasks?"

Q27 · ReentrantLock vs synchronized — when to choose which?

Asked at: Senior rounds

Difficulty: Medium · **Topic:** Concurrency

The question: "When would you use `ReentrantLock` instead of `synchronized`?"

The 30-second answer: When you need: (1) `tryLock` with timeout, (2) interruptible lock acquisition, (3) fairness policy, (4) multiple `Condition` variables (more flexible than one wait/notify), (5) lock metrics (`isHeldByCurrentThread`, `getHoldCount`). Otherwise, `synchronized` is simpler and equally fast.

Step-by-step thought process: 1. "`synchronized` is concise, automatic release on exception, and the JVM heavily optimizes it (biased locking, lightweight locking, lock coarsening). Default choice." 2. "Switch to `ReentrantLock` when you need a capability `synchronized` lacks:" 3. "`tryLock()` or `tryLock(timeout)` — attempt acquisition without blocking forever. Useful for deadlock avoidance." 4. "`lockInterruptibly()` — block but allow interrupt. `synchronized` blocks are not interruptible." 5. "`new ReentrantLock(true)` — fair lock; threads acquire in FIFO order. Slower but prevents starvation." 6. "Multiple `Condition` objects on one lock — replaces the limitation that `synchronized` has only one wait set per monitor. Classic example: producer-consumer with separate `notFull` and `notEmpty` conditions."

Edge cases to mention: - `ReentrantLock` requires manual `lock.unlock()` in a `finally` block — exception-unsafe if you forget - `ReentrantReadWriteLock` allows multiple concurrent readers OR one writer — good for read-heavy data structures - `StampedLock` (Java 8+) adds optimistic reads — faster than `ReentrantReadWriteLock` under heavy read load

What NOT to say: - "ReentrantLock is always faster than synchronized" — wrong; modern JVMs make `synchronized` very fast in uncontended cases - "synchronized is deprecated" — completely false

Common follow-up: "What's the optimistic-read pattern in `StampedLock`?"

Q28 · wait/notify vs Condition — what changed?

Asked at: Mid-level + senior rounds

Difficulty: Medium · **Topic:** Concurrency primitives

The question: "How does `Condition` (from `java.util.concurrent.locks`) improve over `wait/notify`?"

The 30-second answer: Object's `wait/notify` has ONE wait queue per object — you can't selectively wake the right kind of waiter. `Condition` (paired with `Lock`) lets you have MULTIPLE condition variables on one lock — `notFull` and `notEmpty` for a bounded buffer, etc. Plus `await(timeout, unit)` is friendlier than `wait(millis)`.

Step-by-step thought process: 1. "`Object.wait()` / `notify()` / `notifyAll()` work in synchronized blocks. One wait set per object monitor." 2. "Classic problem: bounded buffer with producers waiting on 'not full' and

consumers on 'not empty'. With one wait set, you must call `notifyAll()` and waiters check their condition — wakes up everyone, wasted work." 3. " `Lock` + multiple `Condition` : `final Condition notFull = lock.newCondition(); final Condition notEmpty = lock.newCondition();` .

Producer calls `notFull.await()` — only waits on full-buffer. Consumer signals

`notFull.signal()` after taking — only wakes producers." 4. "Cleaner semantics, less spurious wakeup, better throughput in contention." 5. "Also: `awaitNanos` , `awaitUntil(date)` , `awaitUninterruptibly()` give finer control than `wait()` ."

Edge cases to mention: - Always wait in a loop checking the condition predicate — spurious wakeups exist in both wait/notify and Condition - `signal()` wakes one waiter; `signalAll()` wakes all. Prefer signalAll if uncertain — correctness over performance - BlockingQueue / Semaphore wrap these primitives — usually you should use the higher-level construct

What NOT to say: - "wait/notify and Condition are the same" — they aren't; the multi-condition support is the key win - "Use wait without holding the lock" — runtime exception; both wait and await require holding their respective lock

Common follow-up: "Why do you always wait in a while-loop instead of an if?"

Q29 · Memory model — happens-before relationship

Asked at: Senior rounds at top product cos

Difficulty: Hard · **Topic:** JMM

The question: "What's the happens-before relationship in the Java Memory Model? Give an example."

The 30-second answer: Happens-before is a partial ordering on operations across threads. If A happens-before B, then B sees A's effects (writes visible, reads see the new value). Established by: synchronization (lock release happens-before subsequent acquire), volatile writes (happens-before subsequent volatile reads of the same variable), thread start, thread join, final field freezes.

Step-by-step thought process: 1. "Without happens-before, the JVM and CPU are free to reorder operations within a thread — your reads might see stale values, writes might be invisible to other threads." 2. "The JMM defines specific operations that establish happens-before: synchronized exit → synchronized entry on same monitor; volatile write → subsequent volatile read of same variable; Thread.start() → first action in started thread; final action in thread → Thread.join() in another thread." 3. "Practical example: double-checked locking requires `volatile` on the instance field. Without it: thread A's writes (object construction) might not happen-before thread B's read (the null check), and B might see the reference assigned before the object is fully constructed." 4. " `AtomicInteger.set` and `get` also establish happens-before — that's why Atomic classes are useful for inter-thread communication beyond just numerical operations." 5. "If you have no happens-before relationship, you have no guarantee that one thread's writes are visible to another."

Edge cases to mention: - Final fields have special "freeze" semantics — guaranteed visible to other threads after the constructor returns - Static initializers establish happens-before for all reads of static

fields — useful for safe singletons via inner-class idiom - Volatile is transitive in interesting ways — see "JSR-133 cookbook" for the formal model

What NOT to say: - "Threads share memory, so they see each other's writes" — wrong, that's the whole problem JMM solves - "Volatile makes everything thread-safe" — only for visibility, not for compound operations

Common follow-up: *"How does this connect to the double-checked locking pattern?"*

Q30 · ForkJoinPool and work-stealing

Asked at: Senior backend + parallelism-heavy rounds

Difficulty: Medium-Hard · **Topic:** Parallelism

The question: *"What's ForkJoinPool, and how does work-stealing work?"*

The 30-second answer: ForkJoinPool is a thread pool optimized for divide-and-conquer recursion. Each worker thread has its own deque of tasks. When a worker's deque is empty, it STEALS tasks from another worker's deque. Used internally by parallel streams and CompletableFuture by default.

Step-by-step thought process: 1. *"Designed for tasks that can be split into smaller tasks (`RecursiveTask` / `RecursiveAction`). Submit a big task, it forks subtasks, the pool processes them in parallel, you join the results."* 2. *"Each worker has a local deque (double-ended queue). The worker pushes/pops from one end (LIFO — good cache locality for the just-created subtask)."* 3. *"When a worker's deque is empty, it tries to steal from another worker's deque — but from the OTHER end (FIFO). This minimizes contention with the owning worker."* 4. *"Result: idle threads pick up work automatically; no central queue contention; good cache locality for the busy path."* 5. *"Java's `ForkJoinPool.commonPool()` is shared by parallel streams, `CompletableFuture`, and any code that uses it directly. Be careful about blocking IO in pool threads — it starves the pool."*

Edge cases to mention: - For blocking IO, use `ManagedBlocker` to signal the pool to spawn a replacement thread - Java 21 virtual threads largely replace the need for ForkJoinPool for IO-bound work - Parallel stream performance depends heavily on the Spliterator's split quality — for poorly-splitting sources, parallel is slower

What NOT to say: - "Work-stealing means stealing tasks from the main thread" — wrong, workers steal from each other - "Always use `parallelStream()` for performance" — pool starvation, splitting overhead, and contention often make sequential faster

Common follow-up: *"Why might `parallelStream()` actually be slower than `stream()`?"*

Section D — Java 8+ Functional Programming (lambdas, streams, the new toys)

These are about idiomatic modern Java. Get these wrong and the interviewer pegs you as "stuck in Java 7."

Q31 · Stream API — lazy evaluation explained

Asked at: Common at all levels

Difficulty: Medium · **Topic:** Streams

The question: "Explain lazy evaluation in Java streams. What's the difference between intermediate and terminal operations?"

The 30-second answer: Intermediate operations (`map` , `filter` , `sorted` , etc.) are LAZY — they build up a pipeline but don't execute. Terminal operations (`collect` , `forEach` , `reduce` , `count` , etc.) TRIGGER execution and consume the stream. Until the terminal op runs, nothing in the chain executes — useful for `findFirst` over an infinite stream, etc.

Step-by-step thought process: 1. " `Stream` is a lazy pipeline of operations.

`stream.filter(p).map(f)` doesn't iterate the source — it returns a new `Stream` object describing the intended pipeline." 2. "Terminal operations (`collect` , `count` , `forEach` , `findFirst` , `reduce` , `toArray`) consume the stream and trigger execution. After a terminal op, the stream is closed — you can't use it again." 3. "Laziness enables short-circuiting: `findFirst()` after `filter()` stops as soon as one match is found, even on an infinite stream from `Stream.iterate()` or `Stream.generate()` ." 4. "Laziness also enables fusion: the runtime can combine adjacent map+filter operations into one pass over the source, instead of N intermediate collections." 5. "Caveat: `peek()` is sometimes elided if the runtime determines its result is unused — don't rely on `peek` for side effects you need."

Edge cases to mention: - A Stream can only be consumed once — `s.count()` then `s.collect()` throws `IllegalStateException` - Sort is partially lazy — it's intermediate, but to know the order it must consume the entire upstream before emitting - Parallel streams change execution strategy but the laziness contract is the same

What NOT to say: - "Stream is just a fancy iterator" — misses the lazy + parallel + functional model - "Always materialize to a list with `.collect()` " — fine for finite, fast streams; wrong for infinite or huge ones

Common follow-up: "What's the difference between `map` and `flatMap` ?"

Q32 · Parallel streams — when do they actually help?

Asked at: Senior rounds, often as a trap question

Difficulty: Medium-Hard · **Topic:** Parallelism

The question: "When does `.parallelStream()` actually perform better than `.stream()`, and when is it a trap?"

The 30-second answer: Helps when: (1) CPU-bound work per element is non-trivial, (2) source splits well (ArrayList, ranges — not LinkedList or generated streams), (3) you have many elements (rough rule: at least thousands), (4) you're using the commonPool appropriately. Trap when: tasks are IO-bound, source is hard to split, or the work per element is tiny.

Step-by-step thought process: 1. "`parallelStream()` uses `ForkJoinPool.commonPool()` — by default sized to `Runtime.availableProcessors() - 1` (one worker fewer than logical processors, leaving room for the submitting thread)." 2. "For CPU-bound work, parallelism speedup is roughly bounded by core count, minus overhead." 3. "The source matters: ArrayList and IntStream.range split cleanly (random access, known size). LinkedList must walk to split — defeats the purpose. Stream.iterate is sequential by design." 4. "Per-element work cost matters: simple map operations are dominated by stream overhead, often slower than sequential. Heavy work per element wins." 5. "Element count matters: under ~1000 elements, sequential is usually faster because parallelization setup costs more than the savings." 6. "Trap: IO-bound work in parallel streams clogs the common ForkJoinPool, slowing every other parallel operation in the JVM."

Edge cases to mention: - `forEachOrdered` preserves encounter order but kills parallelism — use `forEach` if order doesn't matter - Custom parallel executor: there's no clean API, but submitting via `ForkJoinPool.submit(() -> stream.parallel().count())` runs in your pool - Java 21 virtual threads + sequential streams is often the better answer for IO-bound work

What NOT to say: - "Parallel is always faster" — common mistake, often false - "Use parallel streams for IO-bound work" — actively bad advice

Common follow-up: "How would you parallelize IO-bound work in Java 21?"

Q33 · Optional — what NOT to do with it

Asked at: Code review style questions, all levels

Difficulty: Medium · **Topic:** Idiomatic Java

The question: "Show me the right and wrong ways to use Optional in Java."

The 30-second answer: Optional was designed for return types where absence is meaningful (e.g. `findById()`). DON'T use it for: fields, parameters, collection types, or as a generic null-replacement. DON'T call `.get()` without checking presence. DO use `.map`, `.flatMap`, `.orElse`, `.ifPresent` for chaining.

Step-by-step thought process: 1. "Optional is a return-type contract — 'this method may not return a value, and you must handle that.' Designed for stream terminals (`findFirst` , `findAny` , `reduce` with no identity) and 'lookup by ID' patterns." 2. "Anti-pattern #1: `Optional` as field. Adds an extra object, doesn't serialize cleanly, and forces every read to unwrap." 3. "Anti-pattern #2: `Optional` as method parameter. Caller can pass `Optional.empty()` OR null OR a real value — three states instead of two. Just overload or use null." 4. "Anti-pattern #3: `optional.get()` without `isPresent()` check — same `NullPointerException`-equivalent, but more verbose." 5. "Idiomatic chaining: `findUser(id).map(User::getEmail).orElse("unknown@example.com")` ." 6. "`ifPresent` for side effects, `orElseGet(supplier)` if the default is expensive to compute."

Edge cases to mention: - `Optional.of(value)` throws if value is null; `Optional.ofNullable(value)` doesn't — pick deliberately - `Optional<List<T>>` is almost always wrong — return empty list instead - Java 10 added `orElseThrow()` (no arg) — throws `NoSuchElementException`, useful when absence is "impossible"

What NOT to say: - "Use Optional everywhere to avoid `NullPointerException`s" — adds boilerplate without solving root cause - "Optional is faster than null checking" — has allocation cost, marginally slower

Common follow-up: "What's `orElseGet` vs `orElse` ?"

Q34 · Method references vs lambdas

Asked at: Common at all levels

Difficulty: Easy-Medium · **Topic:** Functional Java

The question: "When would you use a method reference instead of a lambda?"

The 30-second answer: When the lambda just delegates to an existing method. `s -> s.length()` becomes `String::length` . `(a, b) -> Integer.compare(a, b)` becomes `Integer::compare` . Cleaner to read, no extra closure, sometimes more efficient. Four kinds: static, bound instance, unbound instance, constructor.

Step-by-step thought process: 1. "`Type::staticMethod` — `Integer::parseInt` . Equivalent to `s -> Integer.parseInt(s)` ." 2. "`instance::instanceMethod` — `out::println` . The instance is captured." 3. "`Type::instanceMethod` — `String::length` . The instance becomes the first parameter — `s -> s.length()` ." 4. "`Type::new` — constructor reference. `ArrayList::new` is `() -> new ArrayList()` ." 5. "Use them when the lambda body is just a method call — primarily a readability win. Bytecode-wise, both lambdas and method references compile to `invokedynamic` via `LambdaMetafactory`, so there's no consistent performance difference."

Edge cases to mention: - `this::method` is fine in OO code — captures `this` implicitly, equivalent to `arg -> this.method(arg)` - Ambiguous method references (overloaded methods, multiple matches) need a cast to clarify - A method reference is just sugar — same Functional Interface invariants apply

What NOT to say: - "Method references are faster than lambdas" — sometimes true, but trivial in practice; don't pick on perf grounds - "Use method references everywhere" — sometimes the lambda body adds clarity (renaming variables for readability)

Common follow-up: "What's a `Functional Interface`?"

Q35 · `forEach` vs `for-each` loop — which when?

Asked at: Code review, mid-level

Difficulty: Easy-Medium · **Topic:** Idiomatic Java

The question: "When would you use `Collection.forEach()` vs the enhanced `for-loop`? Are they interchangeable?"

The 30-second answer: Functionally similar for simple iteration. Differences: (1) `forEach` can't break early or use `continue`. (2) Enhanced `for-loop` can mutate the collection only via `Iterator.remove`; `forEach` shouldn't mutate. (3) `forEach` reads better in chains; enhanced `for` is more flexible for complex logic.

Step-by-step thought process: 1. "`for (T item : collection) { ... }` is the enhanced `for-loop` — desugars to an `Iterator`-based loop. Supports `break`, `continue`, `return`." 2. "`collection.forEach(item -> { ... })` is the lambda-based `forEach`. Cannot break — `return` inside the lambda only exits the current iteration (like `continue`)." 3. "For chained stream operations, `stream().filter().map().forEach()` reads cleanly. For standalone iteration with control flow, enhanced `for` is more flexible." 4. "`forEach` doesn't guarantee order on parallel streams — use `forEachOrdered` if you need it." 5. "Concurrent modification: enhanced `for-loop` with `Iterator.remove()` is the safe way to remove during iteration. `forEach` followed by `collection.removeAll(toRemove)` is the lambda equivalent."

Edge cases to mention: - Map's `forEach((k, v) -> ...)` replaces `for (Map.Entry<K,V> e : map.entrySet()) { ... }` — usually cleaner - `Collection.removeIf(predicate)` is the cleanest "remove while iterating" pattern - `forEach` is part of `Iterable`, so any `Iterable` (not just `Collection`) supports it

What NOT to say: - "forEach is faster" — usually the same; for tiny operations, enhanced-`for` can be slightly faster (no lambda boxing) - "Always use `forEach` in modern Java" — sometimes worse readability

Common follow-up: "How would you remove items from a list while iterating?"

Q36 · Collectors — beyond `toList` and `toMap`

Asked at: Common in idiomatic-Java rounds

Difficulty: Medium · **Topic:** Streams

The question: "What Collectors do you reach for beyond `toList()` and `toMap()`?"

The 30-second answer: `groupingBy` for partitioning by key, `partitioningBy` for boolean split, `joining` for string concatenation, `counting` and `summingInt` / `averagingInt` for aggregations. Chain these with downstream collectors for sophisticated reductions in one pass.

Step-by-step thought process: 1. "`groupingBy(classifier)` → `Map<K, List<T>>` . Group employees by department:

`employees.stream().collect(groupingBy(Employee::getDept))` ."

2. "`groupingBy(classifier, downstream)` chains another collector.

`groupingBy(Employee::getDept, counting())` → `Map<String, Long>` of dept →

headcount." 3. "`partitioningBy(predicate)` → `Map<Boolean, List<T>>` . Split adults from minors in one pass." 4. "`joining(delimiter, prefix, suffix)` → `String` . Concatenate names with commas:

`joining(", ", "[", "]")` ." 5. "`teeing(c1, c2, merger)` (Java 12+) feeds the stream to two collectors and combines. E.g. compute min and max in one pass."

Edge cases to mention: - `toMap` throws on duplicate keys unless you supply a merge function:

`toMap(K, V, (existing, replacement) -> existing)` - `groupingBy` uses a `HashMap` by default; use the 3-arg version with `TreeMap::new` for sorted output -

`Collectors.collectingAndThen(downstream, finisher)` post-processes the result — e.g. `collectingAndThen(toList(), Collections::unmodifiableList)` returns an immutable list

What NOT to say: - "Just collect to List and process after" — slower, loses one-pass benefit - "Stream collectors are complicated, just use loops" — defeats the point of streams

Common follow-up: "How would you compute the average salary per department in one pass?"

Q37 · Lambdas vs anonymous inner classes

Asked at: Fresher to mid-level

Difficulty: Easy-Medium · **Topic:** Lambdas

The question: "What's the difference between a lambda and an anonymous inner class?"

The 30-second answer: Both implement a functional interface. Lambda is sugar over

`invokedynamic` (no `.class` file generated), inherits the enclosing `this` , can only access

effectively-final variables. Anonymous inner class generates an extra class, creates a new `this` (and a synthetic reference to the enclosing instance), can access effectively-final variables only too.

Step-by-step thought process: 1. "Lambda: `Runnable r = () -> doSomething();` — no `.class` file, generated at runtime via `invokedynamic`. `this` inside the lambda refers to the enclosing class instance." 2. "Anonymous inner class: `Runnable r = new Runnable() { public void run() { doSomething(); } };` — generates an `EnclosingClass$1.class` , has its own `this` (the inner instance), holds an implicit reference to the enclosing instance via

`EnclosingClass.this` ." 3. "Memory: lambdas without captured state can be a single shared instance (constant). Anonymous classes allocate fresh per invocation." 4. "Variable capture: both can only capture effectively-final local variables — i.e. variables you don't reassign after the lambda/inner-class is defined." 5. "Use lambdas for short functional bodies. Use anonymous inner classes when you need multiple methods, or when you need a fresh `this` (rare)."

Edge cases to mention: - Anonymous classes can extend abstract classes; lambdas can only implement single-abstract-method interfaces - Lambdas don't introduce a new scope — variable shadowing rules are different - Memory footprint: a no-capture lambda is essentially zero allocation per call (cached); an anonymous inner class is at least one allocation

What NOT to say: - "They're the same thing" — close, but `this` semantics differ - "Lambdas are syntactic sugar for inner classes" — close, but invokedynamic doesn't generate an inner class

Common follow-up: "What does 'effectively final' mean?"

Q38 · Functional interface — what makes one?

Asked at: Fresher to mid-level

Difficulty: Easy-Medium · **Topic:** Lambdas

The question: "What's a functional interface? Can you list the most-used ones in `java.util.function`?"

The 30-second answer: A functional interface has exactly ONE abstract method (SAM — Single Abstract Method). Lambdas can be assigned to any functional interface. `@FunctionalInterface` annotation makes the compiler enforce this. Most-used: `Function<T,R>` , `Predicate<T>` , `Consumer<T>` , `Supplier<T>` , `BiFunction<T,U,R>` .

Step-by-step thought process: 1. "A functional interface has exactly one abstract method. Default and static methods don't count." 2. " `@FunctionalInterface` is optional but recommended — the compiler errors if you accidentally add a second abstract method." 3. "Standard library has `Runnable`, `Callable`, `Comparator` (all SAM) plus the `java.util.function` package: `Function`, `Predicate`, `Consumer`, `Supplier`, plus primitive specializations (`IntFunction`, `LongPredicate`, etc.) to avoid boxing." 4. "You can compose them: `Predicate.and()` , `Predicate.or()` , `Function.andThen()` , `Function.compose()` ." 5. "Custom functional interfaces are useful when standard ones don't fit — e.g. a `TriFunction<A, B, C, R>` doesn't exist in the JDK."

Edge cases to mention: - Interfaces with one abstract method + multiple defaults are still functional interfaces (e.g. `Comparator`) - Two interfaces with the same method signature are NOT compatible for lambda assignment — you must pick a concrete one - Method references give you the same functional-interface flexibility as lambdas

What NOT to say: - "Functional interface means it works with streams" — sort of, but the definition is just SAM - Listing 12 functional interfaces when asked to name "a few" — bloat, picks the bad ones

Common follow-up: "What's the difference between Function and Consumer?"

Q39 · default methods — why were they added?

Asked at: Mid-level rounds

Difficulty: Easy-Medium · **Topic:** Java 8 features

The question: "Why were default methods added to interfaces in Java 8? What problem did they solve?"

The 30-second answer: To allow adding methods to existing interfaces without breaking existing implementations. The driver was Stream API: Collection had to gain `stream()`, `parallelStream()`, `forEach()`, `removeIf()` — without `default`, every existing Collection implementation would have had to add these.

Step-by-step thought process: 1. "Pre-Java-8: adding a new abstract method to an interface broke every existing implementation. This blocked Java from evolving its core APIs." 2. "Java 8 added `default` methods — implemented in the interface itself. New implementers don't have to override; existing implementers continue to work." 3. "Used heavily by Java 8 to add Stream support to Collection (`stream()`, `parallelStream()`, `forEach()`), to add `compute*` / `merge` to Map, etc." 4. "Allowed multiple inheritance of behavior (not state) — you can implement two interfaces with default methods that don't conflict. Conflicts must be resolved explicitly via override." 5. "Caveat: default methods on functional interfaces don't change SAM-ness — you can still lambda-assign them."

Edge cases to mention: - Diamond problem: if two interfaces have conflicting defaults, the implementing class MUST override and pick (`InterfaceA.super.method()` or `InterfaceB.super.method()`) - A class's concrete method always wins over interface defaults (no diamond conflict needed) - Static methods on interfaces are also new in Java 8 — bound to the interface, not inherited

What NOT to say: - "Default methods make Java multi-inheritance" — partial truth; state isn't inherited, only behavior - "Use default methods instead of abstract classes" — abstract classes still have their place (state, constructors)

Common follow-up: "What happens if a class implements two interfaces with the same default method?"

Q40 · CompletableFuture exceptionally vs handle

Asked at: Senior backend rounds

Difficulty: Medium · **Topic:** Async error handling

The question: "Difference between `exceptionally`, `handle`, and `whenComplete` on `CompletableFuture`?"

The 30-second answer: `exceptionally(throwable -> fallback)` runs ONLY on exception, recovers with a value. `handle((value, throwable) -> result)` runs on BOTH success and failure, lets you transform either. `whenComplete((value, throwable) -> sideEffect)` runs on both but doesn't transform — for side effects like logging.

Step-by-step thought process: 1. "`exceptionally` : takes a Function from Throwable to T. Invoked only if the future completed exceptionally. Result is a new future with the recovered value." 2. "`handle` : takes a BiFunction of (T, Throwable). Invoked on either outcome — exactly one of value/throwable will be null. Result is a new future with the transformed value." 3. "`whenComplete` : takes a BiConsumer of (T, Throwable). Invoked on either outcome. Result is a future with the SAME value as the original (or rethrows the original exception). For side effects: logging, metrics." 4. "Use `exceptionally` when you have a sensible fallback. Use `handle` when the success and failure paths produce the same type but via different logic. Use `whenComplete` for observation without transformation." 5. "Chain order matters: `future.thenApply(f).exceptionally(g)` — `g` handles exceptions from BOTH the original future AND `f`."

Edge cases to mention: - `exceptionallyCompose` (Java 12+) is the async equivalent of `exceptionally` — recovery via another async call - `whenComplete` rethrows the original exception even if the consumer throws — be careful logging shouldn't mask original errors - Java 12+ `exceptionallyAsync` , `handleAsync` etc. for executor-control versions

What NOT to say: - "They're all the same, just pick one" — they aren't - "Use try-catch on `get()`" — works but blocks; defeats async

Common follow-up: "How would you log an async failure without changing the eventual value?"

Section E — Java 17/21 Hot Topics (the questions most free resources don't cover well)

These are 2025-26 interview questions. Strong answers here mark you as up-to-date — most candidates aren't.

Q41 · Records — what they are and when to use them

Asked at: Razorpay · Cred · Swiggy · senior + mid 2024+

Difficulty: Easy-Medium · **Topic:** Java 16+ features

The question: "What are records in Java 16+? When would you use one instead of a regular class?"

The 30-second answer: Records are compact syntax for immutable data classes. `public record Point(int x, int y) {}` auto-generates: constructor, accessors `x() / y()` , `equals()` , `hashCode()` , `toString()` . Use them for DTOs, value objects, multi-return values — anywhere you'd otherwise write a class with only getters.

Step-by-step thought process: 1. "`public record User(String name, String email, int age) {}`" — one line. Equivalent to a class with: final fields, canonical constructor, accessors (no `get` prefix — just `name()`), `equals` based on all components, `hashCode` based on all components, `toString` listing all components." 2. "Records are implicitly `final`. Cannot extend other classes. Can implement interfaces." 3. "Components are immutable — no setters generated. You CAN add validation in a compact constructor: `public User { if (age < 0) throw new IllegalArgumentException(); }`." 4. "Use cases: REST API request/response DTOs, multi-return values from methods (instead of `Pair` / `Tuple`), map keys (auto-correct `equals`/`hashCode`), event objects." 5. "NOT for: mutable entities (e.g. JPA `@Entity` often needs mutability), classes that need inheritance hierarchies."

Edge cases to mention: - You can override the auto-generated accessor or `toString` / `equals` / `hashCode` if needed — useful for defensive copying of mutable component types - Records can have static factory methods and additional non-component methods - Pattern matching for records (Java 21) lets you destructure: `if (obj instanceof User(var name, var email, var age))`

What NOT to say: - "Records replace Lombok `@Data`" — only for immutable cases; Lombok still wins for mutable entities - "Records can have mutable fields" — wrong, components are always final

Common follow-up: "How does record pattern matching work in Java 21?"

Q42 · Sealed classes — what problem do they solve?

Asked at: Senior rounds at product cos

Difficulty: Medium · **Topic:** Java 17 features

The question: "What are sealed classes (Java 17), and when would you use one over an abstract class?"

The 30-second answer: Sealed classes restrict which classes can extend them. `public sealed class Shape permits Circle, Square, Triangle { }` — only those three classes can extend Shape. Useful for modeling closed domains (like algebraic data types) and for exhaustive pattern matching.

Step-by-step thought process: 1. "Sealed = restricted inheritance. The `permits` clause lists allowed subtypes; the compiler enforces that no other class can extend the sealed type." 2. "Each permitted subtype must be one of: `final` (closed), `sealed` (further restricted), or `non-sealed` (re-opens to any subtype)." 3. "Use when: you have a closed set of variants (`Result = Success | Failure`; `HTTPMethod = GET | POST | PUT | DELETE`; `Shape = Circle | Square | Triangle`)." 4. "Killer feature: pattern matching switch (Java 21) checks exhaustiveness. `switch (shape) { case Circle c -> ...; case Square s -> ...; case Triangle t -> ...; }` — compiler errors if you miss a case." 5. "For libraries: lets you evolve the type hierarchy without breaking users — opposite of abstract classes where anyone could extend unexpectedly."

Edge cases to mention: - Sealed types and permitted subtypes must be in the same module (or same package if not modular) - Sealed interfaces work the same way - Records often work well as the leaf

types of a sealed hierarchy — `sealed interface Result permits Success, Failure { }`
`record Success(String value) implements Result { }`
`record Failure(String reason) implements Result { }`

What NOT to say: - "Sealed is just a fancy abstract" — misses the exhaustiveness + closed-domain win
 - "Use sealed everywhere instead of abstract" — wrong fit for open hierarchies (e.g. plugins, extensions)

Common follow-up: "How does sealed help pattern matching switch?"

Q43 · Pattern matching for switch (Java 21)

Asked at: Razorpay · Swiggy · 2024+ senior rounds

Difficulty: Medium · **Topic:** Java 21 features

The question: "What's pattern matching for switch in Java 21? Give an example with a sealed hierarchy."

The 30-second answer: switch can now match on type patterns and decompose data. Combined with sealed types, you get exhaustiveness checking and concise dispatch — like a lightweight version of pattern matching in Kotlin or Scala. Cleaner than instanceof chains.

Step-by-step thought process: 1. "Pre-Java-21: dispatch by type via if-else-instanceof chains, or visitor pattern. Both verbose." 2. "Java 21 pattern matching for switch lets you switch on Object and match by type, optionally binding the value:" 3.

```
java\nString describe(Shape s) {\n return switch (s) {\n case Circle c -> \"radius \" + c.radius();\n case Square sq -> \"side \" + sq.side();\n case Triangle t -> \"triangle\";\n }; \n}
```

 4. "If Shape is sealed, the compiler enforces exhaustiveness — adding a new permitted subtype causes compile errors anywhere you forgot to handle it. Safe refactoring." 5. "Guard patterns let you add conditions: `case Square sq when sq.side() > 10 -> \"big square\"` ." 6. "Combined with records (Java 21): record patterns destructure — `case Circle(double radius) -> radius * 2` ."

Edge cases to mention: - `case null` is now allowed in switch (Java 21) — gives explicit null handling instead of NPE - Patterns can be nested: `case Container(var content)` etc. - The arrow form (`->`) doesn't fall through; the colon form (`:`) still does — prefer arrow for safety

What NOT to say: - "It's like Kotlin's when" — true but generic; show you've used it - "It's just sugar for instanceof" — partly true but the exhaustiveness + records destructuring is more than sugar

Common follow-up: "How do guards work? Show me a switch where guards matter."

Q44 · Virtual threads (Java 21) — what changes?

Asked at: Razorpay · Cred · Swiggy · Meesho 2024+

Difficulty: Medium-Hard · **Topic:** Java 21 features

The question: "What are virtual threads in Java 21? When do they help vs traditional platform threads?"

The 30-second answer: Virtual threads are lightweight threads managed by the JVM, not the OS. You can have millions of them. Blocking IO unmounts the virtual thread from its carrier (an OS thread), so blocking doesn't waste resources. Huge win for IO-bound server workloads (HTTP handlers, DB calls). No benefit for CPU-bound work.

Step-by-step thought process: 1. "Traditional Java threads (now called 'platform threads') are 1:1 with OS threads. Each takes ~1MB stack + OS scheduling overhead. Practical limit: a few thousand." 2.

"Virtual threads are scheduled by the JVM onto a small pool of platform threads (carriers). They use small stacks (~few KB) that grow as needed." 3. "On blocking IO (file read, socket read, JDBC call), the virtual thread UNMOUNTS from its carrier — the OS thread is free to run another virtual thread. Massive concurrency without thread starvation." 4. "Create one per request:

`Executors.newVirtualThreadPerTaskExecutor()` . Spring Boot 3.2+ supports running Tomcat with virtual threads via a single config flag." 5. "Sweet spot: web servers, microservices, gateway apps — any code that's mostly waiting on IO. NOT useful for CPU-bound work (compression, encryption, ML inference)."

Edge cases to mention: - `synchronized` blocks PIN the virtual thread to its carrier — can't unmount. Use `ReentrantLock` for code that might block while holding a lock. - `ThreadLocal` still works but is more expensive per virtual thread — Java 21's `ScopedValue` is the replacement - Native code (JNI) also pins the virtual thread until the native call returns

What NOT to say: - "Virtual threads make CPU-bound work faster" — false; you can't beat physical cores - "Just use them everywhere" — pinning issues mean you still need to know what blocks

Common follow-up: "What does 'pinning' mean and when does it happen?"

Q45 · Structured concurrency (Java 21 preview)

Asked at: Forward-looking senior rounds

Difficulty: Medium-Hard · **Topic:** Java 21 features

The question: "What's structured concurrency in Java 21, and what problem does it solve?"

The 30-second answer: A new API (preview in Java 21) that treats a group of concurrent tasks as a single unit of work — like try-with-resources for tasks. If the parent scope exits, child tasks are automatically cancelled. Prevents thread leaks and orphaned async work that plagued `CompletableFuture` chains.

Step-by-step thought process: 1. "Problem: `CompletableFuture` chains have no parent-child relationship. If you spawn three async calls and forget to handle one, it runs to completion in the background — silently wasting resources." 2. "Structured concurrency:

```
try (var scope = new StructuredTaskScope.ShutdownOnFailure()) { var f1 = scope.fork(() -> callA()); var f2 = scope.fork(() -> callB()); scope.join();
```

`scope.throwIfFailed(); ... }` 3. "All forks live within the scope. When the scope exits (normally or via exception), any still-running forks are cancelled." 4. "Two strategies built-in: `ShutdownOnFailure` (cancel siblings on first failure — call A and B in parallel, fail fast) and `ShutdownOnSuccess` (cancel siblings on first success — race them, take the winner)." 5. "Together with virtual threads: clean, structured, scalable concurrency that reads top-to-bottom."

Edge cases to mention: - Still in preview as of Java 21 — JEP 453, expected to stabilize in a future release - Custom scopes via subclassing `StructuredTaskScope` for application-specific policies - Designed to work with the existing Thread API — `Thread.currentThread()` works as expected inside forks

What NOT to say: - "It's like `async/await`" — closer, but Java's version is more about lifecycle management - "Use it everywhere" — preview API, lock-in risk if you ship before GA

Common follow-up: "How does this compose with virtual threads?"

Q46 · Sequenced collections (Java 21)

Asked at: Java-21-aware rounds

Difficulty: Easy-Medium · **Topic:** Java 21 features

The question: "What's the new `SequencedCollection` interface in Java 21? What gap does it fill?"

The 30-second answer: A new interface for collections with a defined encounter order. Adds uniform methods: `getFirst()`, `getLast()`, `addFirst()`, `addLast()`, `removeFirst()`, `removeLast()`, `reversed()`. Implemented by `List`, `LinkedHashSet`, `LinkedHashMap`, `Deque`. Replaces the awkward `list.get(0)` / `list.get(list.size()-1)` patterns.

Step-by-step thought process: 1. "Pre-21: getting the first/last element required different code for each collection — `list.get(0)`, `set.iterator().next()`, `deque.getFirst()`. Inconsistent and ugly." 2. "`SequencedCollection` is a new interface (under `Collection` in the hierarchy) with first/last access methods." 3. "Three new interfaces: `SequencedCollection`, `SequencedSet`, `SequencedMap` (separate hierarchies — `Map` isn't a `Collection`)." 4. "`List` and `Deque` now extend `SequencedCollection`; `LinkedHashSet` implements `SequencedSet`; `LinkedHashMap` implements `SequencedMap`. Your existing `ArrayList`, `LinkedList`, `ArrayDeque`, `LinkedHashSet`, `LinkedHashMap` all gain the new methods automatically." 5. "`reversed()` returns a view that iterates in reverse — no copying. Modifications to the view affect the original."

Edge cases to mention: - Not all collections are sequenced — `HashSet` and `HashMap` aren't (no defined order) - `reversed()` on a `List` returns a `List` view; iteration order is reversed but indices still go 0..size - Slight gotcha: `addFirst` on an immutable collection throws `UnsupportedOperationException`

What NOT to say: - "It's just a wrapper around the old methods" — true sort of, but the uniformity is the win - "Use it instead of `List`" — it's not a replacement, just an enhancement

Common follow-up: "How does this change how you'd implement an LRU cache?"

Q47 · Text blocks — what changed in Java 15

Asked at: Easy, modern Java rounds

Difficulty: Easy · **Topic:** Java 15 features

The question: "What's a text block in Java 15? Solve me a problem you'd actually use it for."

The 30-second answer: Triple-quoted multi-line strings. Eliminates the escape-hell of embedded JSON, SQL, HTML in Java source. `"""multi-line text"""`. Automatic indentation normalization based on the closing delimiter position.

Step-by-step thought process: 1. "Pre-15: multi-line strings required `\\n` and `+` concatenation, especially painful for embedded JSON/XML/SQL." 2. "Java 15: text blocks. Open with `\"\"\"`, content on subsequent lines, close with `\"\"\"`. The position of the closing delimiter determines the 'indentation baseline' — leading whitespace up to that column is stripped." 3. "Example: embedding SQL — `String sql = \"\"\"SELECT id, name, email\\n FROM users\\n WHERE active = true\\n \"\"\";` — clean, readable, no concatenation." 4. "Embedded quotes don't need escaping inside text blocks (except triple-quote sequences themselves)." 5. "Newlines are normalized to `\\n` regardless of source file line endings."

Edge cases to mention: - `String.formatted(args)` (Java 15+) is a fluent alternative to `String.format(template, args)` — pairs nicely with text blocks - Embedded expressions are NOT supported (no string interpolation as of Java 21) — template strings were preview in Java 21 + 22 (JEP 430, 459), then withdrawn in Java 23 - Trailing newline can be suppressed with `\\` at the end of a line

What NOT to say: - "Use string concatenation, it's just as readable" — no, embedded JSON in concatenation is unreadable - "Text blocks support string interpolation" — wrong, not yet

Common follow-up: "How would you do string interpolation in Java?"

Q48 · Java 21 ScopedValue vs ThreadLocal

Asked at: Java-21-aware senior rounds

Difficulty: Medium-Hard · **Topic:** Java 21 features

The question: "What's `ScopedValue` (Java 21), and why is it preferred over `ThreadLocal` in modern Java?"

The 30-second answer: `ScopedValue` is an immutable, scoped alternative to `ThreadLocal`. Value is set for the duration of a specific code block (`ScopedValue.runWhere`), then automatically cleared. Solves `ThreadLocal`'s leak problems in virtual-thread / thread-pool environments.

Step-by-step thought process: 1. "ThreadLocal sticks values to threads. In a thread pool or virtual-thread environment, threads are reused, and ThreadLocal values leak across requests unless explicitly cleared." 2. "ScopedValue is bound for the duration of a `run` or `call` block: `ScopedValue.runWhere(USER, currentUser, () -> doWork());`. After the lambda returns, the binding is automatically removed." 3. "Immutable per binding — can't be reassigned mid-scope. Nesting allowed (inner runWhere shadows outer)." 4. "Designed to play well with virtual threads — millions of virtual threads each with bounded scoped values, no leak risk." 5. "Use cases: per-request user/tenant context, transaction context, observability tags."

Edge cases to mention: - Reads are slightly cheaper than ThreadLocal (no map lookup, just a thread-local stack) - Inherits across virtual thread forks within the same structured concurrency scope - Still in preview as of Java 21 (requires `--enable-preview`) — API expected to stabilize in a future release

What NOT to say: - "ScopedValue replaces ThreadLocal everywhere" — gradually; both exist for now - "It's like Spring's RequestContextHolder" — adjacent but lower-level

Common follow-up: "How does this work with virtual threads forking children?"

Q49 · Spring Boot 3 + Java 17 baseline — what changed?

Asked at: Spring backend rounds 2024+

Difficulty: Medium · **Topic:** Spring ecosystem

The question: "You're upgrading a Spring Boot 2 app to Spring Boot 3. What are the major breaking changes?"

The 30-second answer: (1) Java 17 minimum (was 8). (2) Jakarta EE 9 migration — `javax.*` packages renamed to `jakarta.*` across the entire app. (3) New observability stack — Micrometer Observation replaces Sleuth. (4) Native compilation via Spring AOT. (5) Hibernate 6 with new query semantics.

Step-by-step thought process: 1. "Java 17 baseline — your build must use JDK 17+. Many libraries also dropped Java 8/11 support around the same time." 2. "`javax.*` → `jakarta.*` is the most disruptive change. Every `javax.persistence`, `javax.servlet`, `javax.validation` import needs updating. IDE refactor tools help; openrewrite recipe automates much of it." 3. "Spring Sleuth (distributed tracing) is replaced by Micrometer Tracing + Micrometer Observation API." 4. "Spring AOT processing + GraalVM native image support — your jar can build to a standalone binary with sub-second cold start. Useful for serverless." 5. "Hibernate 6 changes: stricter SQL generation, new ID generators, and some JPQL behavior changes — test your queries carefully."

Edge cases to mention: - Spring Security 6 has breaking changes too (lambda-style DSL replaces fluent builder) - Spring Boot 3.2 added virtual thread support — one config flag flips Tomcat to virtual threads - Native compilation has constraints (reflection, dynamic proxies need hints) — not zero-effort

What NOT to say: - "Just bump the version, it'll work" — Jakarta migration alone is a multi-day refactor on real codebases - "Native compilation is faster at runtime" — startup yes, throughput often slightly worse

Common follow-up: *"How would you enable virtual threads in a Spring Boot 3 app?"*

Q50 · The one question they always end with: "What's your favourite Java feature added in the last 3 years?"

Asked at: Closing question, all levels

Difficulty: Soft / signal-question · **Topic:** Self-awareness

The question: *"What's your favourite Java feature added in the last 3 years, and why?"*

The 30-second answer: This is a signal question. They're checking if you actually keep up with the language. Pick something concrete, recent (Java 17 or later), and explain a real use case where it changed your code for the better. Records, pattern matching, virtual threads, sealed classes — any of these work if you can talk about them concretely.

Step-by-step thought process: 1. *"This is a personality/curiosity signal. Wrong answer: 'I don't know, I just write code.' Correct answer: pick one and explain how it affected your work."* 2. *"Strong picks for 2024-26:"* 3. *" **Virtual threads (Java 21)** — 'I migrated our gateway service to virtual threads and our p99 latency dropped 30% because we stopped queuing on the thread pool.'"* 4. *" **Records (Java 16)** — 'I deleted ~2000 lines of DTO boilerplate when we adopted records across our REST layer.'"* 5. *" **Pattern matching for switch (Java 21)** — 'Our event-handling code became exhaustive at compile time — we used to find missed event types in prod.'"* 6. *" **Text blocks (Java 15)** — 'We embedded SQL in code without losing readability or fighting concatenation.'"*

Edge cases to mention: - If the interviewer is on an old Java version, mention it but pivot: "Records are great but I know team's on Java 11 — I've also gotten a lot of value from the Java 8 stream API improvements." - Show curiosity: "I've been playing with structured concurrency in Java 21 — too early to use in production but I like where it's heading." - Don't just list — pick ONE and tell a 30-second story about it

What NOT to say: - "I don't really follow Java updates" — instant downgrade - "Lambdas" (added in Java 8, ~12 years ago) — shows you stopped paying attention - "Generics" (Java 5) — even worse

Common follow-up: *"Have you tried virtual threads in production? What surprised you?"*

What to do next

You've finished the Top 50. If your interview is in the next 5 days:

1. **Re-read Section A (Hot 10) again right now.** These 10 are non-negotiable.
2. **Print 07 Cheatsheet** and carry it to the interview.
3. **The night before:** re-read Section A only. Do NOT add new material at this point.
4. **The morning of:** have breakfast, walk in calm, trust the prep.

If your interview is further out, move on to [02 Top 200 Most-Asked](#) for breadth, or [04 Core Library](#) for depth.

Good luck. You're more prepared than 80% of the candidates in the room.

— The Learn & Excel team